DESIGN AND IMPLEMENTATION OF A SECURE AWS VPN-BASED INTERNAL WEB SERVICE WITH ENCRYPTED STORAGE AND AUTOMATED MONITORING

21IPE333P - Essentials in Cloud and DevOps

Submitted by

PRANAV SAIRAM [RA2311026010195]
AJAYRAJ R [RA2311026010214]
SRIVISHAL S [RA2311026010227]

Under the Guidance of

Dr. RADHIKA R

(Associate Professor, Department of Networking and Communications)

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in Artificial Intelligence and Machine Learning



DEPARTMENT OF NETWORKING AND COMMUNICATIONS
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR- 603 203

NOVEMBER 2026

Department of Networking and Communications
SRM Institute of Science & Technology
Own Work Declaration Form

This sheet must be filled in (each box ticked to show that the condition has been met). It must be signed and dated along with your student registration number and included with all assignments you submit – work will not be marked unless this is done.

To be completed by the student for all assessments

Degree/ Course **:** B.Tech – CSE AIML, 21IPE333P - Essentials in Cloud and DevOps

Student Name **:** PRANAV SAIRAM, AJAYRAJ R, SRIVISHAL S

Registration Number **:** RA2311026010195, RA2311026010214, RA2311026010227

Title of Work **:** Design and Implementation of a Secure AWS VPN-Based Internal Web Service with Encrypted Storage and Automated Monitoring

I / We hereby certify that this assessment complies with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

I / We confirm that all the work contained in this assessment is my / our own except where indicated, and that I / We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Complied with any other plagiarism criteria specified in the Course handbook / University website

I understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:
I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my / our own work, except where indicated by referring, and that I have followed the good academic practices noted above. PRANAV SAIRAM AJAYRAJ R SRIVISHAL S
If you are working in a group, please write your registration numbers and sign with the date for every student in your group.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21IPE333P - Essentials in Cloud and DevOps Project report titled "**Design and Implementation of a Secure AWS VPN-Based Internal Web Service with Encrypted Storage and Automated Monitoring** " is the bonafide work of "**PRANAV SAIRAM [RA2311026010195], AJAYRAJ R [RA2311026010214], SRIVISHAL S [RA2311026010227]"** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE
Dr. RADHIKA R
SUPERVISOR
Associate Professor
DEPARTMENT OF
NETWORKING AND
COMMUNICATIONS

SIGNATURE
DR. M.LAKSHMI
PROFESSOR &HEAD
DEPARTMENT OF
NETWORKING AND
COMMUNICATIONS

ACKNOWLEDGEMENTS

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M,** Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr. M. Pushpalatha**, Professor and Associate Chairperson - CS, School of Computing and **Dr.C. Lakshmi,** Professor and Associate Chairperson -AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, Dr. M. Lakshmi, Professor and Head, Department of Networking and Communications, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We register our immeasurable thanks to our Faculty Advisor, Department of Networking and Communications, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, Dr. RADHIKA R, Associate Professor, Department of Networking and Communications, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under her mentorship. She provided us with the freedom and support to explore the research topics of our interest. Her passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff members of the Department of Networking and Communications, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement

Authors

ABSTRACT

Organisations increasingly host internal tools such as document stores, dashboards and administration portals in the public cloud. When these services are exposed to the internet, even behind a login page, they become targets for credential stuffing, vulnerability scanning and data exfiltration. This project designs and implements a secure internal web service on Amazon Web Services (AWS) that is reachable only through an AWS Client VPN connection authenticated with mutual TLS, stores all data encrypted under a customer-managed key, and monitors and repairs itself automatically.

The service, a document vault called Secure Vault, runs on an Amazon Linux 2023 EC2 instance in a Virtual Private Cloud (VPC) that has only private subnets, with no internet gateway, no NAT gateway, no public IP address and no SSH access. AWS APIs are reached privately through VPC endpoints. A private certificate authority issues per-user client certificates, and revoked users are blocked through a certificate revocation list. One AWS KMS customer-managed key encrypts the EBS volume, the S3 bucket, CloudWatch Logs, CloudTrail, SNS and SSM parameters. The S3 bucket policy permits object access only through the VPC endpoint, so leaked credentials cannot be used to read data from the internet.

Twelve CloudWatch alarms, an in-VPC synthetic health check, log metric filters, VPC Flow Logs, CloudTrail and a budget provide continuous monitoring. An EventBridge-triggered Lambda function automatically revokes any security-group rule opened to the world. The whole environment is defined in Terraform and checked by a continuous integration pipeline. Verification showed that the Terraform configuration is valid, all 13 unit tests pass, the shell scripts are free of shellcheck warnings and 326 of 334 Checkov policy checks pass, with the remaining eight findings analysed in this report. The estimated running cost is about USD 4.35 per day, and the stack can be destroyed and rebuilt in about 15 minutes.

TABLE OF CONTENTS

ABSTRACT.....**v**
TABLE OF CONTENTS.....**vi**
LIST OF FIGURES.....**viii**
LIST OF TABLES.....**ix**
ABBREVIATIONS.....**x**

CHAPTER NO. TITLE PAGE NO.

1 INTRODUCTION.....**1**
1.1 Introduction To Project.....1
1.2 Problem Statement And Description.....2
1.3 Motivation.....3
1.4 Sustainable Development Goal Of The Project.....4
2 LITERATURE SURVEY.....**6**
2.1 Overview Of The Research Area.....6
2.2 Existing Models And Frameworks.....7
2.3 Limitations Identified From Literature Survey (Research Gaps).....8
2.4 Research Objectives.....9
2.5 Product Backlog (Key User Stories With Desired Outcomes).....10
2.6 Plan Of Action (Project Road Map).....11
3 SPRINT PLANNING AND EXECUTION METHODOLOGY.....**12**
3.1 Sprint I: Secure Network Foundation, PKI And VPN Access.....12
3.1.1 Objectives With User Stories Of Sprint I.....12
3.1.2 Functional Document.....12
3.1.3 Architecture Document.....14
3.1.4 Outcome Of Objectives / Result Analysis.....16
3.1.5 Sprint Retrospective.....16
3.2 Sprint II: Secure Application, Encrypted Storage And Delivery Pipeline.....16
3.2.1 Objectives With User Stories Of Sprint II.....16
3.2.2 Functional Document.....17
3.2.3 Architecture Document.....18
3.2.4 Outcome Of Objectives / Result Analysis.....20
3.2.5 Sprint Retrospective.....20
3.3 Sprint III: Automated Monitoring, Alerting And Self-Healing.....20

3.3.1 Objectives With User Stories Of Sprint III....20
3.3.2 Functional Document....21
3.3.3 Architecture Document....22
3.3.4 Outcome Of Objectives / Result Analysis....24
3.3.5 Sprint Retrospective....24
4 RESULTS AND DISCUSSIONS....25
4.1 Project Outcomes (Performance Evaluation, Comparisons, Testing Results)....25
5 CONCLUSION AND FUTURE ENHANCEMENT....30
5.1 Conclusion....30
5.2 Future Enhancement....30
REFERENCES....32
APPENDIX....35
A CODING....35

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	Conceptual overview of the secure internal web service	2
2.1	Project road map across twelve weeks	11
3.1	Private PKI and certificate lifecycle	14
3.2	Network architecture of the isolated VPC	15
3.3	Request path from the user to encrypted storage	19
3.4	Continuous integration pipeline and immutable deployment flow	19
3.5	Monitoring, alerting and self-healing flow	23
3.6	Sequence of automatic remediation by the sg-guard function	23
4.1	Summary of automated verification results	26
4.2	Breakdown of estimated daily running cost	29

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Comparison of existing remote-access models for internal applications	8
2.2	Research gaps and the response of this project	9
2.3	Product backlog: key user stories with desired outcomes	10
2.4	Sprint plan and deliverables	11
3.1	Sprint I user stories	12
3.2	Sprint I functional requirements	13
3.3	Security group design	15
3.4	Sprint I outcomes	16
3.5	Sprint I retrospective	16
3.6	Sprint II user stories	17
3.7	Secure Vault HTTP interface	17
3.8	Sprint II outcomes	20
3.9	Sprint II retrospective	20
3.10	Sprint III user stories	21
3.11	CloudWatch alarms and automated responses	21
3.12	Sprint III outcomes	24
3.13	Sprint III retrospective	24
4.1	Static verification results	25
4.2	Unit test results	26
4.3	Analysis of the eight failed Checkov checks	27
4.4	Evaluation of controls against threats	27
4.5	Automated deployment acceptance tests (smoke-test.sh)	28
4.6	Comparison with alternative architectures	28
4.7	Estimated running cost (us-east-1, while deployed)	29

ABBREVIATIONS

ACM	AWS Certificate Manager
AL2023	Amazon Linux 2023
API	Application Programming Interface
AWS	Amazon Web Services
AZ	Availability Zone
CA	Certificate Authority
CI	Continuous Integration
CIDR	Classless Inter-Domain Routing
CMK	Customer-Managed Key
CRL	Certificate Revocation List
CSP	Content Security Policy
CSRF	Cross-Site Request Forgery
DNS	Domain Name System
EBS	Elastic Block Store
EC2	Elastic Compute Cloud
ENI	Elastic Network Interface
HSTS	HTTP Strict Transport Security
HTTPS	Hypertext Transfer Protocol Secure
IaC	Infrastructure as Code
IAM	Identity and Access Management
IGW	Internet Gateway
IMDSv2	Instance Metadata Service Version 2
KMS	Key Management Service
mTLS	Mutual Transport Layer Security
NAT	Network Address Translation
PKI	Public Key Infrastructure
S3	Simple Storage Service
SDG	Sustainable Development Goal
SG	Security Group
SNS	Simple Notification Service
SSE	Server-Side Encryption
SSM	AWS Systems Manager
SSRF	Server-Side Request Forgery
TLS	Transport Layer Security
VPC	Virtual Private Cloud
VPCE	VPC Endpoint
VPN	Virtual Private Network
WSGI	Web Server Gateway Interface

CHAPTER 1

INTRODUCTION

1.1 Introduction To Project

Cloud computing has changed how organisations build and run software. Instead of buying servers, teams rent compute, storage and networking from providers such as Amazon Web Services (AWS) and describe the whole environment in code. The same flexibility that makes the cloud attractive also makes it easy to expose a service to the internet by mistake. A single security-group rule, a public S3 bucket or an instance with a public IP address is enough to make private data reachable from anywhere in the world. Many organisations therefore need a dependable pattern for internal services: applications that employees use every day but that should never be visible from the public internet.

This project, titled **"Design and Implementation of a Secure AWS VPN-Based Internal Web Service with Encrypted Storage and Automated Monitoring"**, builds such a pattern from first principles. The deliverable is a working internal web application, a document vault called **Secure Vault**, together with all the infrastructure, security controls, monitoring and automation needed to run it safely. Users upload, list, download and delete files through a browser. The application stores the files in Amazon S3 and shows, on a live security page, how each file and the server itself are protected.

The defining property of the system is that it has **no internet-facing surface**. The application runs on an EC2 instance in a Virtual Private Cloud (VPC) that has only private subnets. There is no internet gateway, no NAT gateway, no public IP address and no SSH port. The only way in is AWS Client VPN, which authenticates every user with an individual X.509 client certificate issued by the project's own certificate authority (mutual TLS). AWS services such as S3, Systems Manager and CloudWatch are reached through VPC endpoints, so traffic between the server and AWS never crosses the internet.

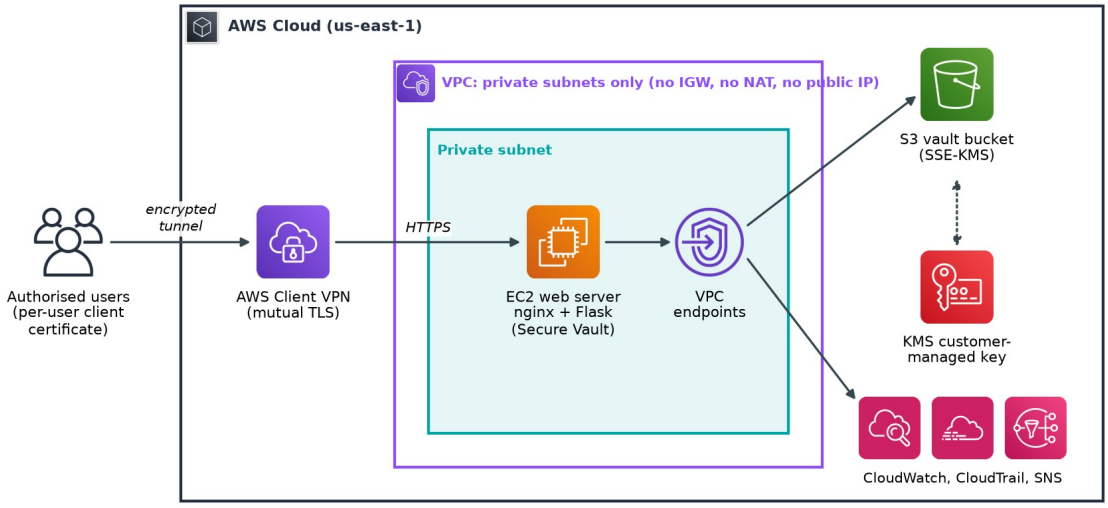


Fig 1.1: Conceptual overview of the secure internal web service

As Fig 1.1 shows, the design combines three concerns that are usually handled separately. **Network isolation** keeps attackers away from the server. **Encryption** protects data in transit and at rest with a customer-managed AWS KMS key, so that a stolen disk snapshot or bucket object is unreadable without the key. **Automated monitoring and self-healing** detect failures and misconfigurations and, where it is safe, fix them without human intervention. For example, a Lambda function automatically revokes any security-group rule that opens a port to the whole internet.

The project also applies DevOps practices throughout. Every resource is defined in Terraform [1], so the environment can be created, reviewed, destroyed and rebuilt from version control. The application is packaged as an offline, reproducible bundle, and each new version replaces the server instead of modifying it in place (immutable deployment). A continuous integration pipeline checks formatting, validates the Terraform configuration, scans it against security policies with Checkov, lints the Python code, runs unit tests with mocked AWS services and checks the shell scripts with shellcheck.

1.2 Problem Statement And Description

Internal applications are often deployed with the same pattern as public websites: a load balancer or server with a public address, protected only by an application login. This leaves the service exposed to automated scanning, brute-force attacks and exploitation of unpatched vulnerabilities, and a single weak password or software flaw can expose sensitive documents. The zero trust guidance published by NIST observes that a network location alone should not be trusted, yet exposing a service to everyone is the opposite extreme [2]. The cost of getting this wrong is high: industry studies continue

to report average breach costs of several million US dollars per incident [3].

At the same time, a secure design must remain practical for a small team. Commercial zero trust platforms and dedicated security operations staff are beyond the budget of many small organisations and student projects. Under the cloud shared responsibility model, AWS secures the underlying infrastructure but the customer remains responsible for network configuration, identity, encryption settings and monitoring [4]. Most real-world cloud incidents arise from mistakes in exactly these customer-managed layers.

The problem addressed by this project can therefore be stated as follows: **design and implement an internal web service on AWS that (i) is unreachable from the public internet and accessible only to individually authenticated users, (ii) encrypts all stored data and all traffic with keys under the owner's control, (iii) prevents stolen cloud credentials from being used to read the data, (iv) detects outages, attacks and misconfigurations automatically and repairs the common ones, and (v) can be reproduced, audited and torn down entirely from code at a predictable, low cost.**

The scope covers a single AWS region (us-east-1) and a single internal application. The system must work without any outbound internet access from the server, which rules out installing software from public package repositories at boot time. Administrators must be able to manage the server without SSH keys or open ports. Every security-relevant event, including VPN connections, API calls and object reads, must be logged and retained in encrypted form.

1.3 Motivation

The first motivation is practical. Remote and hybrid work has made it normal for employees to access internal systems from home networks and mobile connections. A VPN with strong, per-user certificate authentication gives these users secure access while keeping the service itself invisible. Certificate-based mutual TLS is far more resistant to phishing and password reuse than a username and password alone, and individual certificates can be revoked the moment a laptop is lost or an employee leaves.

The second motivation is educational. The course Essentials in Cloud and DevOps covers virtual networks, identity and access management, storage, monitoring, infrastructure as code and continuous delivery. These topics are often practised in isolation. This project brings them together in one system where each decision affects the others. For example, removing the NAT gateway improves

isolation, but it means the server cannot download packages from the internet, which in turn drives the design of an offline application bundle and the use of the S3 gateway endpoint for operating-system repositories.

The third motivation is operational maturity. Security is not a one-time configuration but a continuous process. Studies of site reliability engineering emphasise that systems should alert on symptoms that matter to users and automate repetitive responses [5]. By building synthetic checks, alarms, audit trails and automatic remediation into the infrastructure from the start, the project demonstrates how a small team can achieve a level of operational assurance that would otherwise require dedicated staff.

Finally, cost matters. Cloud resources are billed by the hour, and secure architectures are sometimes assumed to be expensive. A goal of this project is to show that a strongly isolated design can run for about USD 4.35 per day, and that the entire environment can be destroyed when it is not needed and recreated in about 15 minutes from code.

1.4 Sustainable Development Goal Of The Project

The United Nations 2030 Agenda defines seventeen Sustainable Development Goals (SDGs) [6]. This project contributes mainly to the following goals:

- **SDG 9 - Industry, Innovation and Infrastructure (Target 9.1 and 9.c):** the project builds resilient and secure digital infrastructure. Its reusable, open infrastructure-as-code pattern lowers the cost of secure cloud adoption for small organisations, educational institutions and start-ups.
- **SDG 16 - Peace, Justice and Strong Institutions (Target 16.10):** protecting information from unauthorised access supports public access to information while safeguarding fundamental freedoms and privacy. Encryption, strict access control and complete audit trails help institutions handle sensitive records responsibly and build trust.
- **SDG 12 - Responsible Consumption and Production (Target 12.2):** the design avoids idle infrastructure. It uses a single small instance, no NAT gateway, and an environment that is destroyed when not in use, which reduces wasted compute and the associated energy use.
- **SDG 8 - Decent Work and Economic Growth (Target 8.2):** automation of monitoring, alerting and remediation increases productivity by letting a small team operate a secure service without manual round-the-clock supervision.

By showing that strong security can be achieved with managed services, automation and modest cost, the project supports the broader aim of making secure digital infrastructure accessible to organisations of every size.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview Of The Research Area

The project draws on four related research and practice areas: secure remote access and network isolation, cryptographic protection of data, infrastructure as code with DevOps practices, and observability with automated incident response. This section summarises each area and the published guidance that shaped the design.

Secure remote access. The traditional enterprise model places trusted services inside a network perimeter and lets remote users join that perimeter through a VPN. Google's BeyondCorp work showed the weaknesses of relying on the perimeter alone and moved trust decisions to the identity of the user and the device [7]. NIST SP 800-207 formalised these ideas as zero trust architecture, in which every access request is authenticated, authorised and encrypted regardless of network location [2]. A practical outcome of this literature is that a VPN remains useful as an isolation layer, provided that it uses strong per-user authentication and is combined with controls that do not assume the internal network is safe.

Cloud security frameworks. The AWS Well-Architected Framework security pillar recommends a strong identity foundation, traceability, security at all layers, automated security best practices, protection of data in transit and at rest, and preparation for security events [8]. The CIS AWS Foundations Benchmark translates these principles into concrete, checkable controls such as enabling CloudTrail in all regions, enabling key rotation and blocking public S3 access [9]. NIST SP 800-53 provides the broader control catalogue from which many of these checks are derived [10].

Infrastructure as code and DevOps. Infrastructure as code treats servers, networks and policies as versioned software artefacts [11]. Continuous delivery adds automated build, test and deployment pipelines so that every change is verified before it reaches production [12]. The DevOps Handbook emphasises fast feedback, small batches and shared responsibility between development and operations [13]. In a security context, these practices make the security configuration reviewable, repeatable and testable, which is the basis of policy-as-code tools.

Observability and automated response. Site reliability engineering literature distinguishes between symptom-based alerting, which tells operators that users are affected, and cause-based

alerting, which helps diagnosis [5]. NIST SP 800-92 provides guidance on generating, protecting and retaining security logs [14]. Cloud providers now offer event-driven services that can react to configuration changes within seconds, enabling automated remediation of well-understood problems.

2.2 Existing Models And Frameworks

Several established models exist for giving employees access to internal applications in the cloud. Each makes different trade-offs between security, operational effort and cost. The most relevant models are described below and compared in Table 2.1.

Public endpoint with application login. The application is placed behind an internet-facing load balancer, often with a web application firewall and single sign-on. This is simple for users but exposes the service and its software stack to the entire internet. Any vulnerability in the web server, framework or authentication flow can be exploited remotely, and the OWASP Top 10 lists many such classes of flaw [15].

Bastion host with SSH tunnelling. Administrators connect to a hardened jump server in a public subnet and forward ports to private servers. The bastion itself is internet-facing, requires SSH key management and patching, and does not provide a convenient path for non-technical users of a web application.

Self-managed VPN server. An OpenVPN or WireGuard server runs on an EC2 instance in a public subnet [16]. This offers strong encryption and certificate authentication but the team must patch, scale and monitor the VPN server, and it becomes a single point of failure.

Managed client VPN. AWS Client VPN is a managed OpenVPN-compatible service that supports mutual TLS certificate authentication, Active Directory or SAML-based authentication, authorization rules, connection logging to CloudWatch and certificate revocation lists [17]. The service is highly available and requires no server maintenance.

Identity-aware proxy (zero trust access). Services such as identity-aware proxies authenticate every request against an identity provider and device posture before forwarding it to a private application [2,7]. This model provides the strongest per-request control but typically requires an enterprise identity provider, device management and commercial licensing.

Table 2.1: Comparison of existing remote-access models for internal applications

Model	Internet exposure	User authentication	Operational effort	Suitability
Public endpoint + login	Full (application is public)	Password / SSO	Low	Poor for sensitive internal data
Bastion + SSH tunnel	Bastion host is public	SSH keys	Medium (patching, keys)	Administrators only
Self-managed VPN on EC2	VPN server is public	Certificates	High (patching, HA)	Good security, high effort
AWS Client VPN (mTLS)	Managed endpoint only	Per-user certificates + CRL	Low (managed)	Strong, practical (chosen)
Identity-aware proxy	Managed proxy only	SSO + device posture	Medium to high, licensing	Strongest, highest cost

For data protection, the main existing approaches are provider-managed encryption (for example SSE-S3, where AWS manages the keys), customer-managed keys in AWS KMS, and client-side encryption. Provider-managed keys protect against physical media theft but give the customer no independent control or audit of key usage. Customer-managed KMS keys add an independent key policy, CloudTrail logging of every use and the ability to disable the key, which follows the key management guidance of NIST SP 800-57 [18,19]. S3 bucket policies can further restrict access by source VPC endpoint using the `aws:SourceVpce` condition [20].

For private access to cloud APIs, the two existing options are a NAT gateway, which gives private servers outbound internet access, and VPC endpoints (AWS PrivateLink), which expose specific AWS services inside the VPC without any internet path [21]. For server administration, AWS Systems Manager Session Manager provides an IAM-authenticated, fully logged shell without opening inbound ports [22]. For the instance metadata service, IMDSv2 requires session tokens and mitigates server-side request forgery attacks that target cloud credentials [23].

For monitoring and audit, AWS CloudTrail records API activity [24], Amazon CloudWatch collects metrics and logs and evaluates alarms [25], and EventBridge routes events to automated handlers. Policy-as-code scanners such as Checkov evaluate Terraform configurations against hundreds of security rules before deployment [26], and libraries such as moto allow AWS-dependent code to be unit tested without a real account [27].

2.3 Limitations Identified From Literature Survey (Research Gaps)

The survey shows that the individual building blocks are mature and well documented. However, several gaps appear when a small team tries to combine them into one working system.

Table 2.2 summarises these gaps and how this project responds to each.

Table 2.2: Research gaps and the response of this project

Gap identified	Why it matters	Response in this project
Reference architectures usually include a NAT gateway or public subnet	Any internet egress path weakens isolation and enables data exfiltration	Fully private VPC; VPC endpoints for every AWS API; S3 gateway endpoint for OS packages
Encryption guidance stops at enabling SSE	Leaked credentials can still read encrypted objects from anywhere	Bucket policy denies object access unless it arrives through the project VPC endpoint
VPN guides focus on connectivity, not revocation and logging	Lost devices and departing users must be cut off at once	Per-user certificates, scripted revocation, CRL import and failed-connection alarm
Monitoring is configured after deployment, by hand	Gaps and drift are common; alerts arrive late	Alarms, filters, dashboard and budget defined in the same Terraform code
Misconfigurations are detected but not corrected	Exposure persists until a human reacts	Event-driven Lambda revokes world-open security-group rules automatically
Security posture is hard to demonstrate	Auditors and assessors need evidence	Live /api/security endpoint, smoke-test script and policy-as-code scan

2.4 Research Objectives

Based on the problem statement and the gaps above, the project has the following objectives:

- **O1 - Network isolation:** deploy the web service in a VPC with private subnets only, with no internet gateway, NAT gateway, public IP address or SSH access, and reach AWS APIs only through VPC endpoints.
- **O2 - Strong user authentication:** provide remote access exclusively through AWS Client VPN using mutual TLS with per-user certificates issued by a private CA, with scripted issuance, revocation and connection logging.
- **O3 - Encryption everywhere:** encrypt all data at rest (EBS, S3, logs, audit trail, notifications and configuration parameters) with a single customer-managed KMS key with automatic rotation, and enforce TLS 1.2 or higher for all traffic.
- **O4 - Exfiltration resistance:** ensure that stored objects can be accessed only from inside the VPC, even by a principal holding valid AWS credentials.
- **O5 - Automated monitoring and self-healing:** detect host failures, application outages, latency, resource exhaustion, errors, VPN abuse and security configuration changes, notify the operator, and automatically repair failed hosts and world-open firewall rules.

- **O6 - DevOps delivery:** define all infrastructure as code, deploy immutable application bundles, and verify every change with automated formatting, validation, policy scanning, linting and unit testing.
- **O7 - Cost control:** keep the running cost below USD 5 per day with budget alerts, and allow complete teardown and rebuild from code.

2.5 Product Backlog (Key User Stories With Desired Outcomes)

The requirements were captured as user stories for three roles: the end user of the vault, the administrator who operates the platform, and the security auditor who must verify it. Table 2.3 lists the key product backlog items with their priority and the outcome that defines completion.

Table 2.3: Product backlog: key user stories with desired outcomes

ID	User story	Priority	Desired outcome (acceptance)
US1	As an employee, I want to reach the vault only over the VPN so that it is invisible to outsiders.	High	URL does not resolve or connect without VPN; loads with VPN
US2	As an administrator, I want to issue and revoke per-user VPN certificates.	High	make client / make revoke work; revoked user is rejected
US3	As an employee, I want to upload, list, download and delete files in a browser.	High	All four operations work; files stored in S3
US4	As a data owner, I want all stored data encrypted with a key I control.	High	S3 objects show aws:kms with the project CMK; EBS encrypted
US5	As a security auditor, I want stolen AWS keys to be useless for reading files from outside.	High	Bucket policy denies object access not via the VPC endpoint
US6	As an administrator, I want shell access without SSH keys or open ports.	Medium	SSM Session Manager shell works; port 22 closed
US7	As an operator, I want an email when the app is down, slow or failing.	High	Synthetic check and alarms notify within minutes
US8	As a security officer, I want world-open firewall rules removed automatically.	High	sg-guard revokes the rule and sends an alert
US9	As an administrator, I want to ship a new app version safely and reproducibly.	Medium	New bundle replaces the instance; no manual steps
US10	As a developer, I want every change checked automatically.	Medium	CI runs fmt, validate, Checkov, ruff, pytest, shellcheck
US11	As the budget owner, I want to be warned before costs overrun.	Medium	Budget email at 80% actual and 100% forecast
US12	As an auditor, I want every API call and object access recorded.	High	CloudTrail multi-region trail with S3 data events, validated logs

2.6 Plan Of Action (Project Road Map)

The project was organised as a twelve-week plan using an agile, sprint-based approach. Two weeks of requirements analysis and literature survey were followed by three development sprints, a verification phase and a final phase for the deployment demonstration and report. Each sprint ended with a working, verifiable increment and a retrospective. The road map is shown in Fig 2.1 and the sprint goals in Table 2.4.

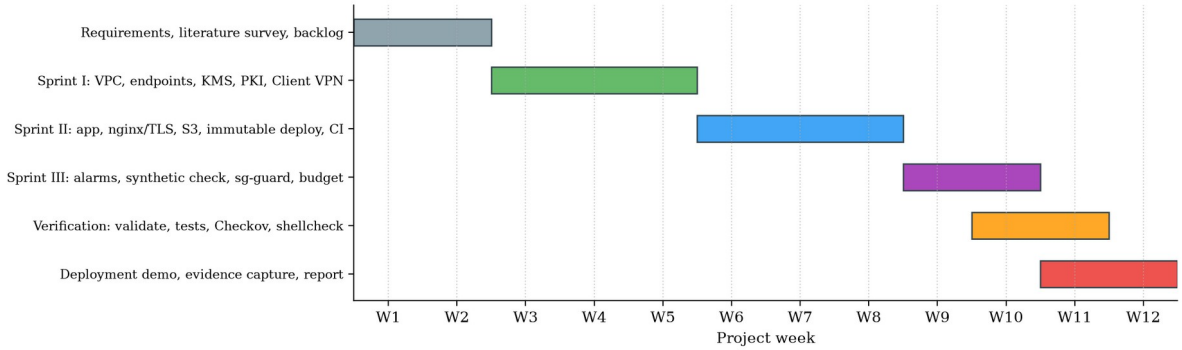


Fig 2.1: Project road map across twelve weeks

Table 2.4: Sprint plan and deliverables

Phase	Weeks	Goal	Key deliverables
Inception	1-2	Understand the problem and design	Literature survey, backlog, architecture draft, cost estimate
Sprint I	3-5	Secure network foundation and VPN access	vpc.tf, kms.tf, vpn.tf, pki.sh, make-ovpn.sh
Sprint II	6-8	Secure application, encrypted storage and delivery	Flask app, nginx, systemd, storage.tf, compute.tf, bundle, CI
Sprint III	9-10	Automated monitoring, alerting and self-healing	monitoring.tf, lambdas.tf, security.tf, dashboard, budget
Verification	10-11	Prove correctness and security	validate, unit tests, Checkov, shellcheck, smoke-test.sh
Closure	11-12	Demonstrate and document	Demo script, evidence, report

CHAPTER 3

SPRINT PLANNING AND EXECUTION METHODOLOGY

Development followed a lightweight Scrum process adapted for a small team. Each sprint began with planning, in which user stories were selected from the product backlog and broken into tasks. Work was tracked on a task board, and every completed task had to pass the automated checks before it was considered done. Each sprint ended with a review of the working increment and a retrospective. The definition of done for every story was: code committed, Terraform formatted and valid, unit tests passing where applicable, and the security impact documented in the architecture notes.

3.1 Sprint I: Secure Network Foundation, PKI And VPN Access

3.1.1 OBJECTIVES WITH USER STORIES OF SPRINT I

The goal of Sprint I was to build the isolated network, the encryption key and the authenticated remote-access path, so that later sprints could deploy the application into an environment that was already secure. The sprint addressed objectives O1, O2 and O3. The selected user stories are listed in Table 3.1.

Table 3.1: Sprint I user stories

ID	User story	Tasks
US1	Reach the service only over the VPN	Private VPC and subnet; no IGW/NAT; Client VPN endpoint; split tunnel; private DNS
US2	Issue and revoke per-user certificates	pki.sh: CA, server, web and client certs; revoke; CRL; push-crl; make-ovpn.sh
US4	Encrypt everything with a key I control	KMS CMK with rotation and service-scoped key policy; default EBS encryption
US6	Administer without SSH	Interface endpoints for ssm, ssmmessages, ec2messages
US12	Record network activity	VPC Flow Logs to encrypted CloudWatch Logs; VPN connection logs

3.1.2 FUNCTIONAL DOCUMENT

The functional requirements for Sprint I are summarised in Table 3.2. Each requirement maps to a Terraform resource or script that implements it and can be checked after deployment.

Table 3.2: Sprint I functional requirements

Req.	Requirement	Implementation
FR1.1	The VPC shall contain only private subnets and no route to the internet	aws_vpc, aws_subnet (map_public_ip_on_launch = false), route table with local route only
FR1.2	The default security group shall allow no traffic	aws_default_security_group with no rules
FR1.3	AWS APIs shall be reachable privately	S3 gateway endpoint; interface endpoints ssm, ssmmessages, ec2messages, logs, monitoring
FR1.4	All VPC traffic shall be logged	aws_flow_log, traffic_type ALL, 60 s aggregation, KMS-encrypted log group
FR1.5	Users shall authenticate with individual certificates	Client VPN certificate-authentication with the private CA chain in ACM
FR1.6	Revoked certificates shall be rejected	pki.sh revoke rebuilds the CRL; push-crl imports it into the endpoint
FR1.7	VPN users shall reach only the web service	VPN security group egress only to the app security group on ports 80 and 443
FR1.8	Only VPC traffic shall use the tunnel	split_tunnel = true; DNS points to the VPC resolver
FR1.9	A single CMK shall encrypt all project data	aws_kms_key with rotation, 7-day deletion window and service-scoped statements

The private PKI is implemented in `scripts/pki.sh` using plain OpenSSL so that no additional tooling is needed. The `init` command creates a root CA, a VPN server certificate, a web server certificate for `app.securevpn.internal` and a first client certificate. Keys are RSA 2048-bit, because AWS Client VPN mutual authentication supports RSA keys of 1024 or 2048 bits, and certificates are signed with SHA-256. Certificate extensions follow the X.509 profile in RFC 5280: the CA certificate carries `keyCertSign` and `cRLSign`, server certificates carry the `serverAuth` extended key usage, and client certificates carry `clientAuth` [28]. The `client <name>` command issues a certificate for a new user, `revoke <name>` revokes it and regenerates the CRL, and `push-crl` imports the CRL into the Client VPN endpoint. The certificate lifecycle is illustrated in Fig 3.1. The helper `make-ovpn.sh` produces a ready-to-import `.ovpn` profile that embeds the CA certificate and the user's certificate and key.

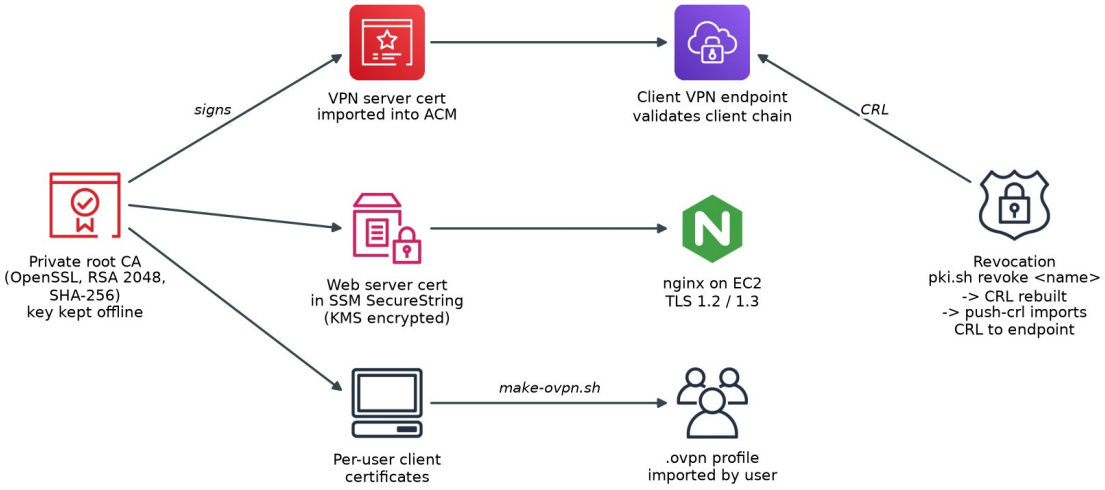


Fig 3.1: Private PKI and certificate lifecycle

3.1.3 ARCHITECTURE DOCUMENT

The network architecture produced in Sprint I is shown in Fig 3.2. The VPC uses the CIDR block 10.0.0.0/16 with a private subnet 10.0.1.0/24. VPN clients receive addresses from a separate pool, 10.100.0.0/22. When a user connects, the Client VPN endpoint creates an elastic network interface (ENI) in the private subnet and forwards the user's traffic from there. DNS queries are sent to the VPC resolver at 10.0.0.2, which answers for the Route 53 private hosted zone `securevpn.internal`.

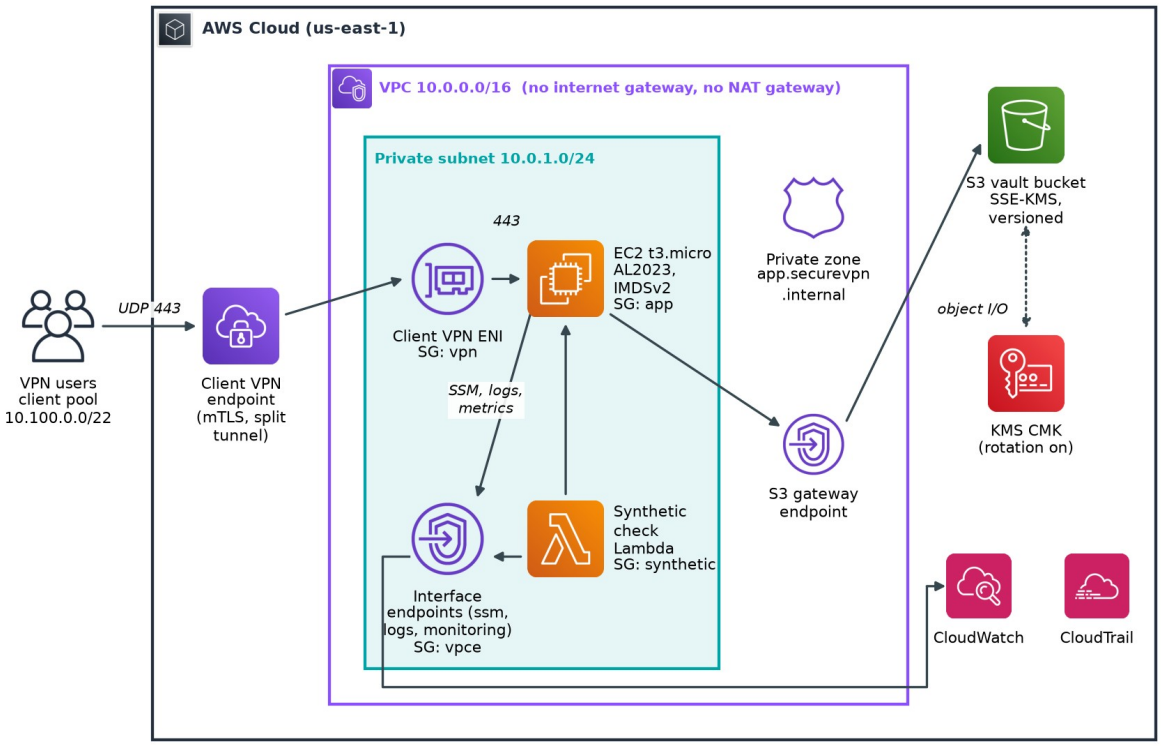


Fig 3.2: Network architecture of the isolated VPC

Access between components is controlled by security groups that reference each other rather than IP ranges. This makes the rules precise and self-documenting. The design of the security groups is given in Table 3.3. No security group in the project allows inbound traffic from 0.0.0.0/0.

Table 3.3: Security group design

Security group	Inbound	Outbound
vpn (Client VPN ENI)	None	TCP 443 and 80 to app SG only
app (EC2 web server)	TCP 443 and 80 from vpn SG; TCP 443 from synthetic SG	TCP 443 to vpce SG; TCP 443 to S3 prefix list
synthetic (health-check Lambda)	None	TCP 443 to app SG and vpce SG
vpce (interface endpoints)	TCP 443 from 10.0.0.0/16	Default
default	None (locked down)	None (locked down)

The KMS key policy grants administrative control to the account, and separately grants CloudWatch Logs, CloudTrail, CloudWatch alarms and EventBridge only the operations they need. Each service statement is constrained by an encryption context or source-account condition, so that, for example, CloudWatch Logs can use the key only for log groups in this account. Automatic key rotation is enabled with a 365-day period, in line with the cryptoperiod guidance of NIST SP 800-57 [18].

3.1.4 OUTCOME OF OBJECTIVES / RESULT ANALYSIS

At the end of Sprint I the foundation was complete. The outcomes against the sprint objectives are summarised in Table 3.4.

Table 3.4: Sprint I outcomes

Objective	Outcome	Evidence
Isolated VPC (O1)	Achieved	vpc.tf defines no IGW, NAT or public subnet; default SG locked
Private AWS API access (O1)	Achieved	1 gateway + 5 interface endpoints with private DNS
Certificate-based VPN (O2)	Achieved	vpn.tf certificate-authentication; pki.sh tested end to end
Revocation (O2)	Achieved	pki.sh revoke + CRL generation verified locally
Customer-managed key (O3)	Achieved	kms.tf with rotation and scoped service statements
Network logging	Achieved	VPC Flow Logs and VPN connection logs to encrypted log groups

The PKI script was exercised end to end: it created the CA, issued the server, web and client certificates, revoked a client and produced a valid CRL. The design associates the Client VPN endpoint with a single subnet, because the endpoint is billed per associated subnet-hour; this halves the largest cost item compared with associating two subnets.

3.1.5 SPRINT RETROSPECTIVE

Table 3.5: Sprint I retrospective

What went well	What did not go well	Actions for the next sprint
Security-group-to-security-group rules made access intent clear	Client VPN accepts only RSA 1024/2048 keys, which was discovered late	Document provider constraints in the code next to the setting
Plain OpenSSL kept the PKI portable	Terraform reads certificate files at plan time, so validation fails before the PKI exists	Generate throwaway certificates in CI before validation
One CMK simplified the key policy	Endpoint hourly cost was higher than expected	Make the number of endpoint AZs a variable (default 1)

3.2 Sprint II: Secure Application, Encrypted Storage And Delivery Pipeline

3.2.1 OBJECTIVES WITH USER STORIES OF SPRINT II

Sprint II delivered the Secure Vault application, its encrypted storage and the delivery pipeline. It addressed objectives O3, O4 and O6. The selected stories are shown in Table 3.6.

Table 3.6: Sprint II user stories

ID	User story	Tasks
US3	Upload, list, download and delete files	Flask app with CSRF protection, filename sanitising, 10 MB limit
US4	Data encrypted with my key	S3 SSE-KMS default encryption with bucket key; encrypted EBS root volume
US5	Stolen keys cannot read data	Bucket policy: TLS only, KMS only, object access only via the VPC endpoint
US6	Administer without SSH	SSM agent and least-privilege instance role; no key pair
US9	Ship new versions safely	Offline wheelhouse bundle; hash-named S3 artefact; user_data replacement
US10	Every change checked	GitHub Actions: fmt, validate, Checkov, ruff, pytest, shellcheck

3.2.2 FUNCTIONAL DOCUMENT

The Secure Vault application is a Flask web application served by gunicorn behind nginx. Its HTTP interface is listed in Table 3.7. All state-changing requests require a CSRF token, uploaded filenames are sanitised to prevent path traversal, and uploads are limited to 10 MB. The application writes structured JSON logs, including an audit event for every upload, download and deletion, which CloudWatch Logs metric filters later use.

Table 3.7: Secure Vault HTTP interface

Method and path	Purpose	Security notes
GET /	List files in the vault	Shows object encryption and version information
POST /upload	Upload a file to S3 under vault/	CSRF token; sanitised name; SSE-KMS with the CMK; 10 MB limit
GET /files/<name>	Download a file	Name validated; path traversal blocked
GET /files/<name>/info	Show object metadata	Displays SSE algorithm and KMS key ARN
POST /files/<name>/delete	Delete a file	CSRF token; previous version retained by versioning
GET /health	Health check for the synthetic monitor	Returns degraded if storage is unreachable
GET /api/security	Live security posture	Bucket encryption, public-access block, versioning, IMDSv2, no public IP

nginx terminates TLS and accepts only TLS 1.2 and TLS 1.3 with forward-secret AEAD cipher suites [29]. Plain HTTP on port 80 is redirected to HTTPS. nginx adds HTTP Strict Transport Security [30], a restrictive Content Security Policy, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, a no-referrer policy and a permissions policy. It also applies per-client rate limiting (20 requests per second with a burst of 40), hides its version, and writes access logs in JSON.

gunicorn listens only on the loopback interface, so the application cannot be reached except through nginx.

The gunicorn service runs as an unprivileged `vault` user under a hardened systemd unit [31]. The unit sets `NoNewPrivileges`, `ProtectSystem=strict`, `ProtectHome`, `PrivateTmp`, `PrivateDevices`, kernel and control-group protections and an empty capability bounding set, and allows writes only to its log directory. Even if an attacker found a flaw in the application, the process could not modify the system or gain privileges.

The storage layer applies defence in depth. The S3 bucket uses SSE-KMS with the project CMK as default encryption, with an S3 bucket key to reduce the number of KMS requests. Block Public Access is fully enabled and object ACLs are disabled. Versioning keeps previous versions for 30 days so that accidental deletions can be undone. The bucket policy contains three deny statements: it rejects any request not made over TLS, rejects uploads that request a weaker encryption algorithm, and rejects every object read, write or delete that does not arrive through the project's S3 gateway endpoint [20]. The only exception is the deploying principal, so that Terraform can upload the application bundle and remove the bucket on teardown.

The EC2 instance requires IMDSv2 with a hop limit of 1, which blocks the class of server-side request forgery attacks that steal instance credentials through the metadata service [23]. Its IAM role grants only what the application needs: list and read-write access under the `vault/` prefix, read access to `artifacts/`, read access to the bucket's security configuration, use of the project key, and read access to the project's SSM parameters. The TLS certificate and private key are delivered as KMS-encrypted SSM SecureString parameters.

3.2.3 ARCHITECTURE DOCUMENT

The request path through the application tier is shown in Fig 3.3. A request from the user's browser travels through the VPN tunnel, is resolved by private DNS, is decrypted by nginx, is forwarded over the loopback interface to gunicorn and Flask, and results in an S3 call through the gateway endpoint. The object is encrypted at rest by S3 using a data key generated by KMS (envelope encryption).

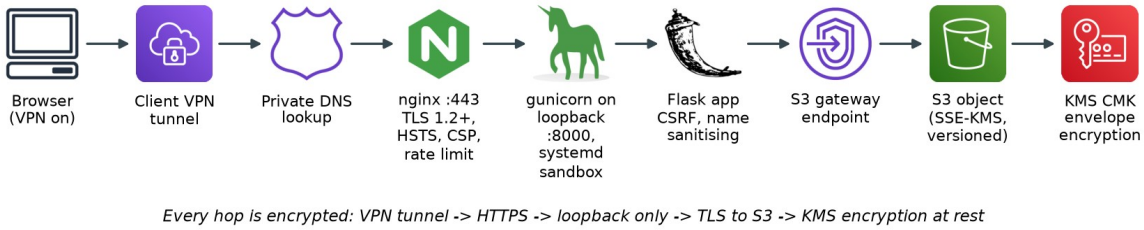


Fig 3.3: Request path from the user to encrypted storage

Because the server has no internet access, it cannot install Python packages from PyPI at boot. The delivery pipeline, shown in Fig 3.4, solves this with an offline bundle. The script `build-bundle.sh` downloads the exact package versions pinned in `app/requirements.lock` as wheels for the instance's Python 3.9 runtime, producing 18 wheels, and packages them with the application code. Terraform uploads the bundle to S3 under a key that contains its SHA-256 hash. Because the hash is passed to the instance's user data and `user_data_replace_on_change` is enabled, any change to the application replaces the instance with a fresh one. This is immutable deployment: servers are never modified in place, so configuration drift cannot accumulate [11,12].

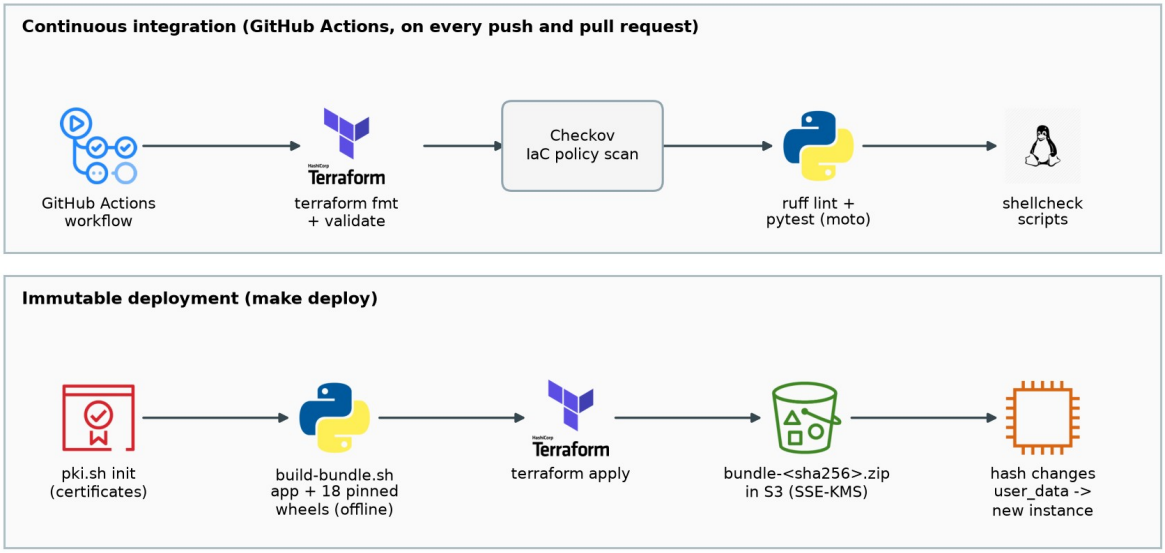


Fig 3.4: Continuous integration pipeline and immutable deployment flow

At boot, the user data script installs nginx from the Amazon Linux repositories, which are served from S3 and therefore reachable through the gateway endpoint. It then fetches the bundle and TLS material, creates a virtual environment from the local wheels, writes the nginx and systemd configuration, configures the CloudWatch agent from an SSM parameter and starts the services. Any

failure is written to a bootstrap log that is monitored by a metric filter.

3.2.4 OUTCOME OF OBJECTIVES / RESULT ANALYSIS

Table 3.8: Sprint II outcomes

Objective	Outcome	Evidence
Working vault application (US3)	Achieved	8 application unit tests pass, including the upload, download and delete round trip
Encryption at rest (O3)	Achieved	Test asserts objects are stored with aws:kms and the configured key
Exfiltration resistance (O4)	Achieved	storage.tf DenyObjectAccessOutsideVpc statement
TLS 1.2+ in transit (O3)	Achieved	nginx.conf ssl_protocols TLSv1.2 TLSv1.3; S3 TLS-only policy
Offline reproducible build (O6)	Achieved	build-bundle.sh produced 18 pinned wheels
Automated checks (O6)	Achieved	CI workflow with four jobs; all checks run locally

The application tests use moto to simulate S3 and KMS, so they verify the real boto3 calls without an AWS account [27]. They confirm that a CSRF-less POST is rejected, that a malicious filename is sanitised, that a path-traversal download is blocked, that deleting a file keeps the previous version and that the security posture endpoint reports the expected configuration.

3.2.5 SPRINT RETROSPECTIVE

Table 3.9: Sprint II retrospective

What went well	What did not go well	Actions for the next sprint
Moto made storage logic testable without cloud cost	The instance runs Python 3.9 while development used 3.12, so wheels had to target 3.9	Pin the target Python version in the bundle script
Immutable deployment removed manual steps	The bucket's VPC-endpoint-only rule also blocked Terraform's own upload	Exempt only the deploying principal ARN
systemd hardening needed no code changes	Provider download for validation hit a local disk quota	Clear caches; install Terraform in the user's home directory

3.3 Sprint III: Automated Monitoring, Alerting And Self-Healing

3.3.1 OBJECTIVES WITH USER STORIES OF SPRINT III

Sprint III made the system observable and self-correcting, and added the audit trail and cost controls. It addressed objectives O5 and O7. The stories are listed in Table 3.10.

Table 3.10: Sprint III user stories

ID	User story	Tasks
US7	Email when the app is down, slow or failing	Synthetic check Lambda; 12 CloudWatch alarms; encrypted SNS topic
US8	World-open firewall rules removed automatically	sg-guard Lambda triggered by EventBridge from CloudTrail
US11	Warning before costs overrun	AWS Budget with 80% actual and 100% forecast notifications
US12	Every API call and object access recorded	Multi-region CloudTrail with log file validation and S3 data events

3.3.2 FUNCTIONAL DOCUMENT

Monitoring combines infrastructure metrics, an end-to-end synthetic check, log-based metrics and audit events. The twelve CloudWatch alarms are listed in Table 3.11. All alarms notify a KMS-encrypted SNS topic that sends email to the operator, and they also notify when they return to the OK state.

Table 3.11: CloudWatch alarms and automated responses

Alarm	Signal and threshold	Response
ec2-system-status-failed	StatusCheckFailed_System > 0	EC2 auto-recover + email
ec2-instance-status-failed	StatusCheckFailed_Instance > 0	Automatic reboot + email
cpu-high	CPUUtilization > 80%	Email
memory-high	mem_used_percent > 85% (CloudWatch agent)	Email
disk-high	disk_used_percent > 80% (CloudWatch agent)	Email
app-unavailable	Synthetic Availability < 1; missing data = breaching	Email
app-latency-high	Synthetic LatencyMs > 2000	Email
nginx-5xx	>= 5 HTTP 5xx responses in 5 minutes	Email
app-errors	>= 5 ERROR log lines in 5 minutes	Email
bootstrap-failed	>= 1 user-data failure	Email
vpn-failed-connections	>= 3 failed VPN connections in 5 minutes	Email
vpc-rejected-packets	>= 200 rejected flows in 5 minutes (possible scan)	Email

The **synthetic check** is a Python 3.12 Lambda function attached to the VPC. Every five minutes it calls `https://app.securevpn.internal/health`, verifies the server certificate against the bundled private CA with a minimum of TLS 1.2, and publishes `Availability` (1 or 0) and `LatencyMs` metrics. Because it runs inside the VPC, it tests the same path that users take, including DNS, security groups, TLS and the application's connection to S3. Missing data is treated as breaching, so the alarm

also fires if the check itself stops running.

The **sg-guard** function implements automated remediation. CloudTrail records every `AuthorizeSecurityGroupIngress` and `ModifySecurityGroupRules` call, and an EventBridge rule invokes the function. The function checks whether the affected security group is tagged with the project tag, lists its rules and identifies any ingress rule open to 0.0.0.0/0 or ::/0. It revokes those rules and publishes a summary to SNS, including who made the change and from which IP address. Groups outside the project are left untouched, which avoids interfering with unrelated resources.

A further EventBridge rule sends an alert whenever security-relevant configuration changes, such as changes to security groups, KMS keys, S3 bucket policies or CloudTrail itself. The **CloudTrail** trail is multi-region, has log file integrity validation enabled, records S3 object-level data events for the vault bucket, and stores its logs in a separate encrypted, versioned bucket [14,24]. Optional GuardDuty findings of severity 4 or above can also be forwarded. Finally, an **AWS Budget** warns at 80% of actual spend and 100% of forecast spend, and a **CloudWatch dashboard** shows all key metrics and alarm states on one screen [25].

3.3.3 ARCHITECTURE DOCUMENT

The flow of signals from sources to automated responses is shown in Fig 3.5. Metrics and logs flow into CloudWatch, where alarms evaluate them. Audit events flow from CloudTrail into EventBridge, where rules route them either to the sg-guard function or directly to the SNS topic. The two EC2 status alarms carry their own repair actions, using the built-in EC2 recover and reboot actions.

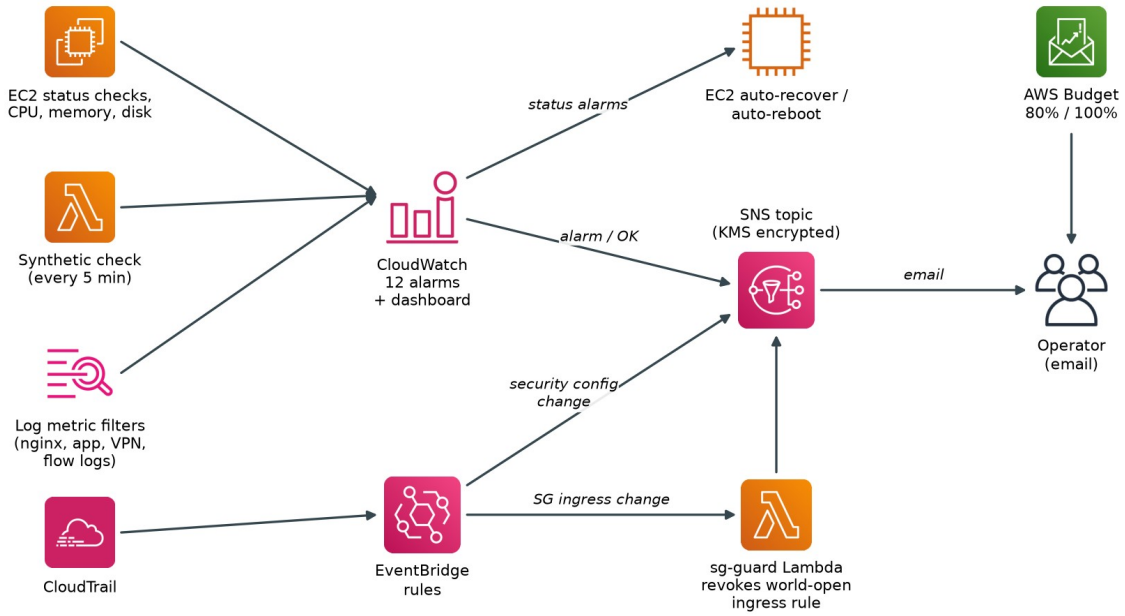


Fig 3.5: Monitoring, alerting and self-healing flow

The interaction sequence for automatic remediation is shown in Fig 3.6. The total delay between the offending API call and the revocation is dominated by CloudTrail event delivery, which is typically within a few minutes. The function is idempotent: if it runs twice for the same event, the second run finds no open rules and does nothing.

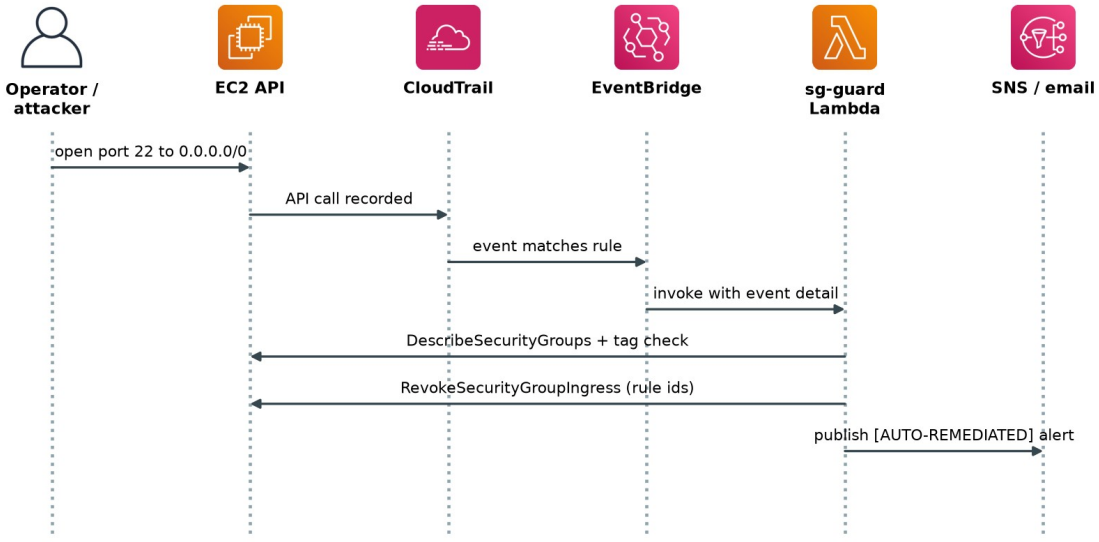


Fig 3.6: Sequence of automatic remediation by the sg-guard function

3.3.4 OUTCOME OF OBJECTIVES / RESULT ANALYSIS

Table 3.12: Sprint III outcomes

Objective	Outcome	Evidence
Outage and latency detection (O5)	Achieved	Synthetic Lambda; 2 unit tests for healthy and unreachable targets
Host self-healing (O5)	Achieved	Status alarms with EC2 recover and reboot actions
Automatic remediation (O5)	Achieved	sg-guard; 3 unit tests incl. revoke-on-project-group-only
Complete audit trail (US12)	Achieved	Multi-region trail, log validation, S3 data events
Cost control (O7)	Achieved	Budget alerts; estimated USD 4.35 per day

3.3.5 SPRINT RETROSPECTIVE

Table 3.13: Sprint III retrospective

What went well	What did not go well	Actions
Config maps for metric filters kept 5 alarms in one block of code	Encrypted SNS topics need explicit KMS permissions for CloudWatch and EventBridge	Add a dedicated key-policy statement with a source-account condition
Treating missing data as breaching caught a silent failure mode	Lambda network interfaces delay teardown by several minutes	Document the expected destroy time
Pure functions in sg-guard were easy to unit test	Checkov flagged several findings that need judgement	Record each accepted finding with a justification

CHAPTER 4

RESULTS AND DISCUSSIONS

4.1 Project Outcomes (Performance Evaluation, Comparisons, Testing Results)

This chapter presents the verification results, analyses the policy-as-code findings, evaluates the design against the threats identified in Chapter 1, compares it with alternative architectures and analyses the cost. All results in Tables 4.1 to 4.3 were produced by running the tools against the final code base.

Static verification. Table 4.1 summarises the automated checks. The Terraform configuration was formatted and validated with Terraform 1.17.1 and the AWS provider. The Python code passed the ruff linter, all shell scripts passed shellcheck, and all 13 unit tests passed. The results are shown graphically in Fig 4.1.

Table 4.1: Static verification results

Check	Tool	Result
Terraform formatting	terraform fmt -check -recursive	Pass (no changes needed)
Terraform validity	terraform validate	Pass: "The configuration is valid"
Policy-as-code scan	Checkov (terraform framework)	326 passed, 8 failed (10 checks excluded with written justification)
Python lint	ruff check	Pass: all checks passed
Unit tests	pytest with moto	13 passed, 0 failed
Shell scripts	shellcheck scripts/*.sh	Pass: no warnings
PKI	scripts/pki.sh init / revoke	CA, server, web and client certificates and CRL created
Offline bundle	scripts/build-bundle.sh	18 pinned wheels packaged

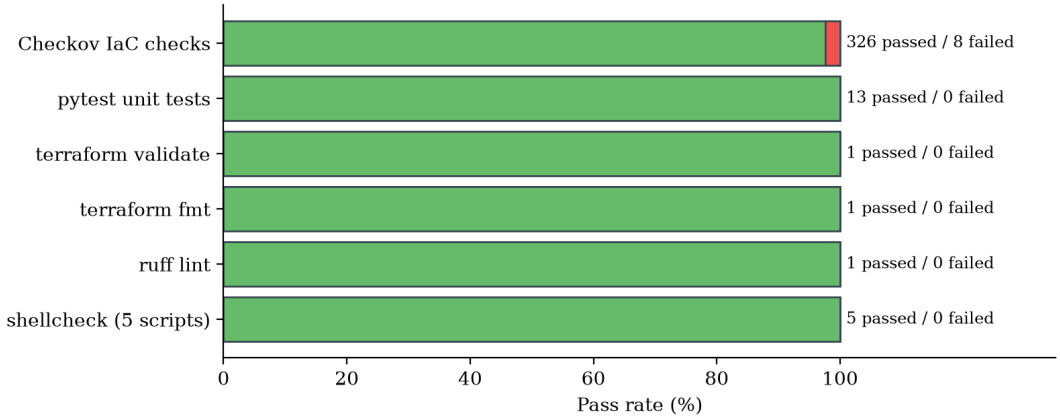


Fig 4.1: Summary of automated verification results

Table 4.2: Unit test results

Test	What it proves	Result
test_health_ok	Health endpoint reports ok	Passed
test_health_degraded_when_storage_unreachable	Storage failure is detected	Passed
test_upload_list_download_roundtrip_uses_kms	Files round-trip and are stored with SSE-KMS	Passed
test_post_without_csrf_token_rejected	CSRF protection works	Passed
test_filename_is_sanitised	Malicious filenames are cleaned	Passed
test_path_traversal_download_blocked	Path traversal is blocked	Passed
test_delete_keeps_previous_version	Versioning allows recovery	Passed
test_security_posture_endpoint	Posture endpoint reports controls	Passed
test_find_open_ingress_rules_only_flags_world_open_ingress	sg-guard flags only world-open ingress	Passed
test_extract_group_id_handles_both_api_shapes	Both EC2 API event shapes handled	Passed
test_sg_guard_revokes_public_rule_on_project_group_only	Remediation limited to project groups	Passed
test_probe_reports_failure_for_unreachable_target	Synthetic check detects outage	Passed
test_probe_reports_success_for_healthy_response	Synthetic check reports healthy app	Passed

Policy-as-code findings. Checkov evaluated 334 checks. Ten further checks were excluded deliberately, each with a written justification in `.checkov.yaml`; for example, S3 server access logging is replaced by CloudTrail S3 data events, and one-year log retention is reduced to 30 days to control cost. The eight failed checks were analysed individually, as shown in Table 4.3. Four are false positives or are inherent to how AWS key policies work, and four are minor hardening opportunities that are recorded as future work.

Table 4.3: Analysis of the eight failed Checkov checks

Check	Resource	Analysis	Disposition
CKV_AWS_260 (port 80 from 0.0.0.0/0)	app_http_from_vpn	Rule source is the VPN security group, not 0.0.0.0/0	False positive
CKV_AWS_356, CKV_AWS_111, CKV_AWS_109	KMS key policy	Resource "*" in a key policy means "this key"; service statements carry conditions	Expected for key policies
CKV2_AWS_34 (SSM parameter encrypted)	cwagent_config	Holds the CloudWatch agent configuration, which contains no secrets	Accept; could use SecureString
CKV_AWS_252 (CloudTrail SNS topic)	aws_cloudtrail.main	Delivery notifications are not needed; EventBridge consumes events	Future hardening
CKV_AWS_394 (pin AZ identity)	availability_zones data	Only the first AZ is used	Future hardening
CKV2_AWS_3 (GuardDuty organisation)	guardduty_detector	Single-account project; GuardDuty optional	Not applicable

Security evaluation against threats. Table 4.4 evaluates the implemented controls against the threats that motivated the project. Each threat is addressed by at least one preventive control and one detective control, giving defence in depth.

Table 4.4: Evaluation of controls against threats

Threat	Preventive control	Detective / corrective control
Internet attacker reaches the server	No IGW/NAT, no public IP, SG ingress only from VPN SG	Flow-log REJECT alarm
Stolen VPN profile	Per-user certificates; CRL revocation	Connection logs; failed-connection alarm
VPN user pivots to other hosts	VPN SG egress only to the app SG	Flow logs
Eavesdropping	mTLS tunnel; TLS 1.2+; S3 TLS-only policy	nginx logs record TLS protocol and cipher
Leaked AWS keys used to read data	Bucket policy: object access only via VPC endpoint	CloudTrail S3 data events
Disk or snapshot theft	CMK encryption of EBS, S3, logs, SNS, SSM	KMS usage in CloudTrail
Accidental or malicious deletion	Versioning, 30-day noncurrent retention	Audit events in app log and CloudTrail
SSH brute force	No SSH; Session Manager only	Session activity in CloudTrail
SSRF credential theft	IMDSv2 required, hop limit 1	-
Port opened to the world	-	sg-guard revokes the rule and alerts
Tampering with security configuration	Least-privilege IAM	EventBridge alert on SG/KMS/policy/trail changes
Application outage	Immutable, hardened deployment	Synthetic check; auto-recover and reboot
Runaway cost	Small instance, single AZ endpoints	Budget alerts

Deployment acceptance tests. The script `scripts/smoke-test.sh` automates the acceptance tests that are run after deployment. It contains 23 checks, summarised in Table 4.5. The AWS posture checks run with or without the VPN; the network checks run while connected; and the `--no-vpn` mode proves that the application is unreachable when disconnected. Together with the demonstration script `scripts/demo.sh`, which triggers a test alarm, an application outage and a world-open security-group rule, these tests provide repeatable evidence for each objective.

Table 4.5: Automated deployment acceptance tests (smoke-test.sh)

Area	Checks
Network isolation	Instance has no public IP; VPC has no internet gateway; app SG has no 0.0.0.0/0 ingress; IMDSv2 enforced
VPN	Endpoint available; certificate (mutual TLS) authentication; connection logging enabled
Encryption and storage	Root EBS encrypted with the CMK; S3 default SSE-KMS; Block Public Access; versioning; KMS rotation; plain-HTTP S3 access denied
Monitoring	CloudTrail logging; alarm topic has a confirmed subscription; synthetic check reports healthy
Over the VPN	Private DNS resolves; HTTPS health passes with CA verification; HTTP redirects with 301; TLS 1.1 refused; HSTS present; app reports SSE-KMS and public access blocked
Without the VPN	Application is not reachable

Comparison with alternative architectures. Table 4.6 compares the implemented design with two common alternatives for the same application. The chosen design removes the public attack surface entirely while keeping operational effort low, at a cost comparable to the alternatives.

Table 4.6: Comparison with alternative architectures

Criterion	Public ALB + login	Bastion + NAT	This project
Internet-reachable components	Load balancer, web app	Bastion, NAT egress	Managed VPN endpoint only
User authentication	Password / SSO	SSH keys	Per-user mTLS certificates
Outbound internet from server	Usually yes	Yes (NAT)	None
Leaked keys can read data	Yes	Yes	No (VPCE-only policy)
Admin access	SSH or SSM	SSH via bastion	SSM only, no open ports
Automatic remediation	Not typical	Not typical	Yes (sg-guard, auto-recover)
Reproducible from code	Varies	Varies	Yes (Terraform, CI)

Cost analysis. The estimated running cost in us-east-1 is given in Table 4.7 and Fig 4.2. The Client VPN subnet association is the largest item at USD 2.40 per day, followed by the five interface

endpoints at USD 1.20 per day. Removing the NAT gateway avoids its hourly and data-processing charges. Because the entire environment is defined in Terraform, it can be destroyed when not in use, so the real monthly cost depends on how many days it is deployed.

Table 4.7: Estimated running cost (us-east-1, while deployed)

Item	Rate	Per day (USD)
Client VPN subnet association	$0.10 per hour	2.40
Client VPN connection (8 hours per day)	$0.05 per hour	0.40
5 interface endpoints in 1 AZ	$0.01 per hour each	1.20
EC2 t3.micro	$0.0104 per hour	0.25
KMS key, Route 53 zone, CloudWatch, S3, Lambda	Usage based	about 0.10
Total		about 4.35

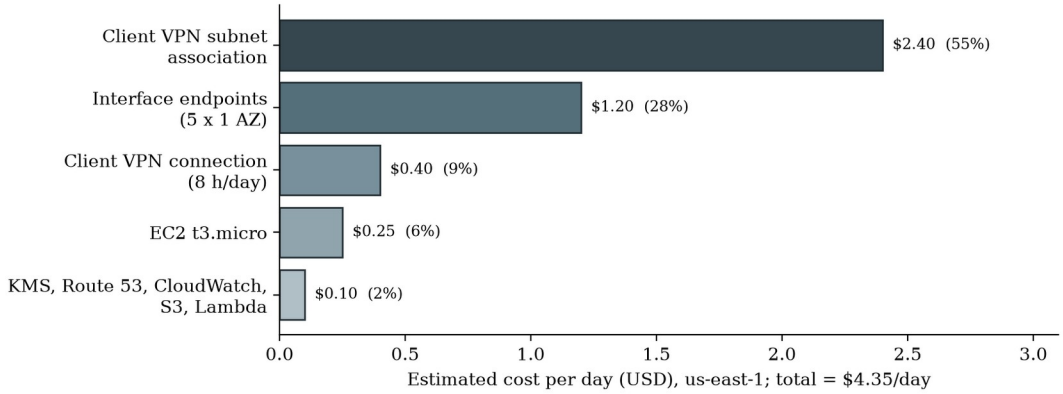


Fig 4.2: Breakdown of estimated daily running cost

Discussion. The results show that each objective is met by concrete, verifiable controls. The most significant result is the combination of network isolation with the VPC-endpoint-only bucket policy: an attacker would need to be inside the VPC, which requires a valid, unrevoked client certificate, before any stolen credential becomes useful. The policy-as-code scan confirmed broad compliance with established benchmarks and highlighted a small number of trade-offs that are documented rather than hidden. The main limitation is availability: a single instance in one Availability Zone is a deliberate cost decision. EC2 auto-recovery covers hardware failure, but not the loss of an entire Availability Zone.

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENT

5.1 Conclusion

This project designed and implemented a secure internal web service on AWS that is reachable only through a certificate-authenticated VPN, encrypts all of its data under a customer-managed key, and monitors and repairs itself automatically. The Secure Vault application runs in a VPC with no internet path in either direction. Users authenticate with individual X.509 certificates that can be revoked at any time. Data is encrypted in transit with TLS 1.2 or higher and at rest with AWS KMS, and stored objects can be read only from inside the VPC, even by someone holding valid AWS credentials.

Operational assurance is built into the infrastructure rather than added afterwards. Twelve alarms, an in-VPC synthetic check, log metric filters, VPC Flow Logs and a multi-region CloudTrail trail give continuous visibility. Failed hosts are recovered or rebooted automatically, and any security-group rule opened to the world is revoked within minutes by an event-driven Lambda function. A budget protects against unexpected cost.

The project also demonstrates DevOps practice. The entire environment is defined in Terraform, application releases are immutable and reproducible from an offline bundle, and every change is verified by a continuous integration pipeline. Verification confirmed a valid Terraform configuration, 13 of 13 passing unit tests, clean linting and shell checks, and 326 of 334 passing Checkov checks, with every remaining finding analysed. The design runs for about USD 4.35 per day and can be rebuilt from code in about 15 minutes, showing that strong cloud security is achievable for small teams at modest cost.

5.2 Future Enhancement

- **Federated authentication:** combine certificate authentication with SAML single sign-on through IAM Identity Center, so that VPN access follows group membership and multi-factor authentication.
- **High availability:** run the application in an Auto Scaling group across two Availability Zones behind an internal Application Load Balancer with AWS WAF, and associate the VPN

endpoint with subnets in both zones.

- **Backup and immutability:** add an AWS Backup plan for EBS and S3 Object Lock in compliance mode for records that must not be altered.
- **Continuous compliance:** enable AWS Config conformance packs and Security Hub to score the environment continuously against the CIS benchmark.
- **Pipeline-driven deployment:** use a GitHub Actions OIDC role to run terraform plan on every pull request and apply approved changes, removing long-lived credentials.
- **Address remaining scan findings:** encrypt the CloudWatch agent parameter, pin Availability Zone identities and add a CloudTrail delivery topic.

REFERENCES

[1] HashiCorp, "Terraform Language Documentation," 2024. [Online]. Available: https://developer.hashicorp.com/terraform/language

[2] S. Rose, O. Borchert, S. Mitchell and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, National Institute of Standards and Technology, Gaithersburg, MD, 2020.

[3] IBM Security and Ponemon Institute, "Cost of a Data Breach Report 2024," IBM Corporation, Armonk, NY, 2024.

[4] Amazon Web Services, "Shared Responsibility Model," AWS Compliance documentation. [Online]. Available: https://aws.amazon.com/compliance/shared-responsibility-model/

[5] B. Beyer, C. Jones, J. Petoff and N. R. Murphy, Eds., Site Reliability Engineering: How Google Runs Production Systems, Sebastopol, CA: O'Reilly Media, 2016.

[6] United Nations General Assembly, "Transforming our World: the 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, 2015.

[7] R. Ward and B. Beyer, "BeyondCorp: A New Approach to Enterprise Security," ;login: The USENIX Magazine, vol. 39, no. 6, pp. 6-11, 2014.

[8] Amazon Web Services, "Security Pillar - AWS Well-Architected Framework," AWS Whitepaper, 2024. [Online]. Available: https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/

[9] Center for Internet Security, "CIS Amazon Web Services Foundations Benchmark," version 3.0.0, 2024.

[10] Joint Task Force, "Security and Privacy Controls for Information Systems and Organizations," NIST Special Publication 800-53 Revision 5, 2020.

[11] K. Morris, Infrastructure as Code: Dynamic Systems for the Cloud Age, 2nd ed., Sebastopol, CA: O'Reilly Media, 2021.

[12] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Boston, MA: Addison-Wesley, 2010.

[13] G. Kim, J. Humble, P. Debois and J. Willis, The DevOps Handbook, Portland, OR: IT Revolution Press, 2016.

[14] K. Kent and M. Souppaya, "Guide to Computer Security Log Management," NIST Special Publication 800-92, 2006.

[15] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: https://owasp.org/Top10/

[16] OpenVPN Inc., "OpenVPN 2.6 Reference Manual," 2023. [Online]. Available:

https://openvpn.net/community-resources/reference-manual-for-openvpn-2-6/

[17] Amazon Web Services, "AWS Client VPN Administrator Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/

[18] E. Barker, "Recommendation for Key Management: Part 1 - General," NIST Special Publication 800-57 Part 1 Revision 5, 2020.

[19] Amazon Web Services, "AWS Key Management Service Developer Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/kms/latest/developerguide/

[20] Amazon Web Services, "Amazon S3 User Guide: Controlling Access from VPC Endpoints with Bucket Policies," 2024. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/example-bucket-policies-vpc-endpoint.html

[21] Amazon Web Services, "AWS PrivateLink Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/vpc/latest/privatelink/

[22] Amazon Web Services, "AWS Systems Manager User Guide: Session Manager," 2024. [Online]. Available: https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html

[23] Amazon Web Services, "Amazon EC2 User Guide: Use the Instance Metadata Service Version 2 (IMDSv2)," 2024. [Online]. Available: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-service.html

[24] Amazon Web Services, "AWS CloudTrail User Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/awscloudtrail/latest/userguide/

[25] Amazon Web Services, "Amazon CloudWatch User Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/

[26] Prisma Cloud (Palo Alto Networks), "Checkov: Policy-as-Code for Infrastructure as Code," documentation, 2024. [Online]. Available: https://www.checkov.io/

[27] Moto contributors, "Moto: Mock AWS Services," documentation, 2024. [Online]. Available: https://docs.getmoto.org/

[28] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley and W. Polk, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile," RFC 5280, IETF, May 2008.

[29] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, IETF, Aug. 2018.

[30] J. Hodges, C. Jackson and A. Barth, "HTTP Strict Transport Security (HSTS)," RFC 6797, IETF, Nov. 2012.

[31] freedesktop.org, "systemd.exec - Execution Environment Configuration," systemd manual

page, 2024. [Online]. Available:
https://www.freedesktop.org/software/systemd/man/systemd.exec.html

APPENDIX

A CODING

This appendix reproduces the most important parts of the implementation. The complete source code, including all Terraform files, scripts, tests and documentation, is in the project repository.

A.1 Isolated VPC and VPC endpoints (terraform/vpc.tf, excerpt)

```
# Isolated VPC: private subnets only. There is no internet gateway and no NAT
# gateway, so nothing in this VPC can reach or be reached from the internet.
# AWS APIs are reached privately through VPC endpoints.

resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = { Name = "${local.name}-vpc" }
}

# Lock down the default security group so nothing can accidentally use it.
resource "aws_default_security_group" "default" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "${local.name}-default-locked" }
}

resource "aws_subnet" "private" {
  count = length(local.azs)

  vpc_id                  = aws_vpc.main.id
  cidr_block              = cidrsubnet(var.vpc_cidr, 8, count.index + 1)
  availability_zone       = local.azs[count.index]
  map_public_ip_on_launch = false

  tags = { Name = "${local.name}-private-${local.azs[count.index]}", Tier = "private" }
}

resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "${local.name}-private-rt" }
}

resource "aws_route_table_association" "private" {
  count          = length(aws_subnet.private)
  subnet_id      = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private.id
}
```

A.2 Client VPN with mutual TLS (terraform/vpn.tf, excerpt)

```
resource "aws_ec2_client_vpn_endpoint" "main" {
  description            = "${local.name} client VPN"
  server_certificate_arn = aws_acm_certificate.vpn_server.arn
  client_cidr_block      = var.client_vpn_cidr
  vpc_id                 = aws_vpc.main.id
  security_group_ids     = [aws_security_group.vpn.id]
```

```
  dns_servers            = [cidrhost(var.vpc_cidr, 2)]
  split_tunnel           = true
  transport_protocol     = "udp"
  vpn_port               = 443
  session_timeout_hours  = var.vpn_session_timeout_hours
  self_service_portal    = "disabled"

  # Server and client certificates are issued by the same private CA, so the
  # server certificate's chain is used to validate client certificates.
  authentication_options {
    type                       = "certificate-authentication"
    root_certificate_chain_arn = aws_acm_certificate.vpn_server.arn
  }

  connection_log_options {
    enabled               = true
    cloudwatch_log_group  = aws_cloudwatch_log_group.vpn.name
    cloudwatch_log_stream = aws_cloudwatch_log_stream.vpn.name
  }

  client_login_banner_options {
    enabled     = true
    banner_text = "Authorized use only. All connections to ${local.name} are logged and
monitored."
  }

  tags = { Name = "${local.name}-client-vpn" }
}

# One subnet association keeps cost down (billed per associated subnet-hour).
# Associating a subnet also adds the route to the VPC CIDR automatically.
resource "aws_ec2_client_vpn_network_association" "main" {
  client_vpn_endpoint_id = aws_ec2_client_vpn_endpoint.main.id
  subnet_id              = aws_subnet.private[0].id
}

resource "aws_ec2_client_vpn_authorization_rule" "vpc" {
  client_vpn_endpoint_id = aws_ec2_client_vpn_endpoint.main.id
  target_network_cidr    = var.vpc_cidr
  authorize_all_groups   = true
  description            = "Certificate-authenticated clients may reach the VPC"
}
```

A.3 Bucket policy denying access outside the VPC endpoint (terraform/storage.tf, excerpt)

```
  # Object data is only readable/writable from inside the VPC. Leaked
  # credentials used from the internet cannot touch the files.
  statement {
    sid    = "DenyObjectAccessOutsideVpc"
    effect = "Deny"
    actions = [
      "s3:GetObject",
      "s3:GetObjectVersion",
      "s3:PutObject",
      "s3:DeleteObject",
      "s3:DeleteObjectVersion",
    ]
    resources = ["${aws_s3_bucket.data.arn}/*"]
    principals {
      type        = "*"
      identifiers = ["*"]
    }
    condition {
      test     = "StringNotEquals"
      variable = "aws:SourceVpce"
      values   = [aws_vpc_endpoint.s3.id]
    }
    condition {
```

```
      test     = "ArnNotEquals"
      variable = "aws:PrincipalArn"
      values   = [local.deployer_arn]
    }
  }
}
```

A.4 Hardened EC2 instance (terraform/compute.tf, excerpt)

```
resource "aws_instance" "app" {
  ami                         = nonsensitive(data.aws_ssm_parameter.al2023_ami.value)
  instance_type               = var.instance_type
  subnet_id                   = aws_subnet.private[0].id
  vpc_security_group_ids      = [aws_security_group.app.id]
  iam_instance_profile        = aws_iam_instance_profile.app.name
  associate_public_ip_address = false
  ebs_optimized               = true
  monitoring                  = false

  metadata_options {
    http_endpoint               = "enabled"
    http_tokens                 = "required"
    http_put_response_hop_limit = 1
    instance_metadata_tags      = "disabled"
  }

  root_block_device {
    volume_type           = "gp3"
    volume_size           = 10
    encrypted             = true
    kms_key_id            = aws_kms_key.main.arn
    delete_on_termination = true
  }

  user_data_replace_on_change = true
  # ... user_data, tags and dependencies omitted ...
}
```

A.5 Automatic remediation Lambda (lambdas/sg_guard/handler.py)

```
"""Auto-remediation: revoke world-open ingress on project security groups.

Triggered by EventBridge when CloudTrail records AuthorizeSecurityGroupIngress
or ModifySecurityGroupRules. If the affected group is tagged
Project=<PROJECT_TAG> and now has an ingress rule open to 0.0.0.0/0 or ::/0,
the rule is revoked and an alert is published to SNS.
"""

import json
import logging
import os

import boto3

logger = logging.getLogger()
logger.setLevel(logging.INFO)

PROJECT_TAG = os.environ.get("PROJECT_TAG", "securevpn")
SNS_TOPIC_ARN = os.environ.get("SNS_TOPIC_ARN", "")
OPEN_CIDRS = {"0.0.0.0/0", "::/0"}

ec2 = boto3.client("ec2")
sns = boto3.client("sns")
```

```
def extract_group_id(detail: dict) -> str | None:
    params = detail.get("requestParameters") or {}
    if params.get("groupId"):
        return params["groupId"]
    modify = params.get("ModifySecurityGroupRulesRequest") or {}
    return modify.get("GroupId")


def find_open_ingress_rules(rules: list[dict]) -> list[dict]:
    return [
        rule
        for rule in rules
        if not rule.get("IsEgress")
        and (rule.get("CidrIpv4") in OPEN_CIDRS or rule.get("CidrIpv6") in OPEN_CIDRS)
    ]


def is_project_group(group: dict) -> bool:
    tags = {tag["Key"]: tag["Value"] for tag in group.get("Tags", [])}
    return tags.get("Project") == PROJECT_TAG


def handler(event, context):
    detail = event.get("detail", {})
    group_id = extract_group_id(detail)
    if not group_id:
        logger.info("No security group id in event; ignoring")
        return {"action": "ignored", "reason": "no-group-id"}

    groups = ec2.describe_security_groups(GroupIds=[group_id])["SecurityGroups"]
    if not groups or not is_project_group(groups[0]):
        logger.info("Group %s is not tagged Project=%s; ignoring", group_id, PROJECT_TAG)
        return {"action": "ignored", "reason": "not-project-group", "group_id": group_id}

    rules = ec2.describe_security_group_rules(
        Filters=[{"Name": "group-id", "Values": [group_id]}]
    )["SecurityGroupRules"]
    open_rules = find_open_ingress_rules(rules)
    if not open_rules:
        return {"action": "none", "group_id": group_id}

    rule_ids = [rule["SecurityGroupRuleId"] for rule in open_rules]
    ec2.revoke_security_group_ingress(GroupId=group_id, SecurityGroupRuleIds=rule_ids)

    actor = (detail.get("userIdentity") or {}).get("arn", "unknown")
    summary = {
        "action": "revoked",
        "group_id": group_id,
        "rule_ids": rule_ids,
        "rules": [
            {
                "protocol": r.get("IpProtocol"),
                "from_port": r.get("FromPort"),
                "to_port": r.get("ToPort"),
                "cidr": r.get("CidrIpv4") or r.get("CidrIpv6"),
            }
            for r in open_rules
        ],
        "actor": actor,
        "source_ip": detail.get("sourceIPAddress"),
    }
    logger.warning(json.dumps(summary))

    if SNS_TOPIC_ARN:
        sns.publish(
            TopicArn=SNS_TOPIC_ARN,
            Subject=f"[AUTO-REMEDIATED] Public ingress revoked on {group_id}",
```

```
            Message=json.dumps(summary, indent=2),
        )
    return summary
```

A.6 Synthetic health check Lambda (lambdas/synthetic_check/handler.py)

```
"""In-VPC synthetic check for the internal web service.

Calls the /health endpoint over HTTPS, verifying the server certificate
against the project's private CA (bundled as ca.crt), and publishes
Availability (1/0) and LatencyMs metrics to CloudWatch.
"""

import json
import logging
import os
import ssl
import time
import urllib.error
import urllib.parse
import urllib.request

import boto3

logger = logging.getLogger()
logger.setLevel(logging.INFO)

TARGET_URL = os.environ.get("TARGET_URL", "https://app.securevpn.internal/health")
NAMESPACE = os.environ.get("METRIC_NAMESPACE", "securevpn/Synthetics")
TIMEOUT_SECONDS = float(os.environ.get("TIMEOUT_SECONDS", "8"))
CA_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ca.crt")

cloudwatch = boto3.client("cloudwatch")



def build_ssl_context(ca_file: str = CA_FILE) -> ssl.SSLContext:
    context = ssl.create_default_context(cafile=ca_file)
    context.minimum_version = ssl.TLSVersion.TLSv1_2
    return context



def probe(url: str, context: ssl.SSLContext, timeout: float) -> dict:
    """Return {'ok': bool, 'latency_ms': float, 'status': int|None, 'error': str|None}."""
    started = time.monotonic()
    status = None
    error = None
    try:
        request = urllib.request.Request(url, headers={"User-Agent":
"securevpn-synthetic/1.0"})
        with urllib.request.urlopen(request, timeout=timeout, context=context) as response:
            status = response.status
            body = json.loads(response.read().decode("utf-8"))
            if status != 200 or body.get("status") != "ok":
                error = f"unhealthy response: status={status} body={body}"
    except urllib.error.HTTPError as exc:
        status = exc.code
        error = f"HTTP {exc.code}"
    except Exception as exc:  # noqa: BLE001 - any failure means "down"
        error = f"{type(exc).__name__}: {exc}"
    latency_ms = (time.monotonic() - started) * 1000
    return {"ok": error is None, "latency_ms": round(latency_ms, 1), "status": status, "error":
error}



def handler(event, context):
    result = probe(TARGET_URL, build_ssl_context(), TIMEOUT_SECONDS)
```

```
    endpoint = urllib.parse.urlparse(TARGET_URL).hostname
    dimensions = [{"Name": "Endpoint", "Value": endpoint}]

    cloudwatch.put_metric_data(
        Namespace=NAMESPACE,
        MetricData=[
            {"MetricName": "Availability", "Dimensions": dimensions, "Value": 1 if result["ok"]
else 0, "Unit": "Count"},
            {"MetricName": "LatencyMs", "Dimensions": dimensions, "Value":
result["latency_ms"], "Unit": "Milliseconds"},
        ],
    )

    log = logger.info if result["ok"] else logger.error
    log(json.dumps({"target": TARGET_URL, **result}))
    return result
```

A.7 Hardened systemd unit (app/deploy/vault.service)

```
[Unit]
Description=Secure Vault internal web service (gunicorn)
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=vault
Group=vault
WorkingDirectory=/opt/vault
EnvironmentFile=/etc/vault.env
ExecStart=/opt/vault/venv/bin/gunicorn --workers 2 --threads 4 --bind 127.0.0.1:8000 --access-
logfile - --forwarded-allow-ips 127.0.0.1 wsgi:app
Restart=always
RestartSec=3

# Hardening
NoNewPrivileges=true
PrivateTmp=true
PrivateDevices=true
ProtectSystem=strict
ProtectHome=true
ProtectKernelTunables=true
ProtectKernelModules=true
ProtectControlGroups=true
RestrictSUIDSGID=true
LockPersonality=true
RestrictRealtime=true
CapabilityBoundingSet=
ReadWritePaths=/var/log/vault

[Install]
WantedBy=multi-user.target
```

A.8 Continuous integration workflow (.github/workflows/ci.yml)

```
name: ci

on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read
```

```
jobs:
  terraform:
    name: Terraform fmt / validate
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.9.8
      # validate needs the files that file() reads; CI uses throwaway certs
      # and an empty bundle (nothing is deployed from CI).
      - name: Generate throwaway PKI and bundle placeholder
        run: |
          scripts/pki.sh init
          mkdir -p build/bundle && echo placeholder > build/bundle/README
      - run: terraform -chdir=terraform fmt -check -recursive
      - run: terraform -chdir=terraform init -backend=false
      - run: terraform -chdir=terraform validate

  security-scan:
    name: Checkov IaC scan
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: bridgecrewio/checkov-action@v12
        with:
          directory: terraform
          config_file: .checkov.yaml
          framework: terraform

  python:
    name: Lint + unit tests
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - run: pip install -r requirements-dev.txt
      - run: ruff check app lambdas tests
      - run: python -m pytest -q tests

  frontend:
    name: Frontend lint + build
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: fe
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "24"
          cache: npm
          cache-dependency-path: fe/package-lock.json
      - run: npm ci
      - run: npm run lint
      - run: npm run build

  shellcheck:
    name: Shellcheck scripts
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: shellcheck scripts/*.sh
```